

UNIVERSIDAD DEL NORTE

Departamento de Ingenieria de Sistemas y Computacion

Estructura de Datos II

ALCALDE DIGITAL

Videojuego educativo sobre el uso responsable de las redes sociales

DOCUMENTACION - PRIMERA ENTREGA

Lenguaje: Python

Interfaz: PyQt6

Estructura: Arbol Binario de Busqueda (ABB)

Integrantes: Sebastian Andres Anaya Ojeda
Andres Santiago Mendible Remon
Adrian Jose Martinez Martinez
Roger Alberto Daza Polo

Asignatura: Estructura de Datos II

Docente: Katherine Ospino Bermejo

Fecha: Septiembre de 2026

Repositorio: <https://github.com/SAAO1/CIVICA>

Barranquilla, Colombia

1 Descripción de la solución

Alcalde Digital es un videojuego educativo desarrollado en Python y PyQt6. La historia se desarrolla en Ciudad Nova durante un periodo electoral y utiliza la red social ficticia Civitas para representar la circulación de publicaciones, rumores y noticias. Los jugadores toman decisiones frente a la información que reciben, como compartir, verificar, ignorar o reportar.

La primera entrega se concentra en demostrar el uso funcional de un **Arbol Binario de Búsqueda (ABB)** dentro del videojuego. Cada nodo representa una publicación y utiliza su identificador como llave.

El documento del laboratorio establece que esta primera entrega evalúa cinco aspectos: **arboles**, **pertinencia de la estructura**, **operaciones y recorridos**, **interfaz gráfica preliminar** y **relación del árbol con la lógica del juego**. [1]



Figura 1: Pantalla de inicio del videojuego *Alcalde Digital*.

2 Pertinencia del Arbol Binario de Búsqueda

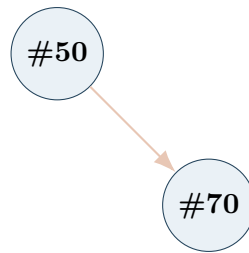
2.1 Problema que resuelve

Durante la partida aparecen publicaciones que deben ser gestionadas por el sistema. Se necesita una estructura que permita incorporar una publicación, localizarla cuando el jugador interactúa con ella y retirarla cuando corresponda.

El ABB permite resolver este problema utilizando el `id` de la publicación como llave. La regla es:

$$\text{ID izquierdo} < \text{ID del nodo} < \text{ID derecho}$$

Por ejemplo, si se insertan las publicaciones #50 y #70, la segunda queda a la derecha porque $70 > 50$.



2.2 Por que se escogio esta estructura

El laboratorio indica que las estructuras deben solucionar un problema real dentro de la logica del videojuego y no utilizarse solamente como decoracion. Tambien propone como usos posibles de los arboles la clasificacion de publicaciones, las jerarquias de informacion, las consecuencias de las decisiones y los arboles de decisiones. [1]

En nuestro proyecto, el ABB se utiliza para **organizar y procesar las publicaciones de Civitas**. Por esta razon, las operaciones del arbol tienen consecuencias visibles dentro de la partida.

2.3 Variante utilizada

La variante utilizada es un **Arbol Binario de Busqueda no autoequilibrado**. En esta primera entrega no se utiliza AVL ni otra variante autoequilibrada.

3 Representacion de los datos

3.1 Nodo

Cada nodo contiene una publicacion y dos referencias: una hacia el hijo izquierdo y otra hacia el hijo derecho.

Listing 1: Clase Nodo.

```
1 class Nodo:
2     def __init__(self, publicacion):
3         self.publicacion = publicacion
4         self.izquierda = None
5         self.derecha = None
```

3.2 Publicacion

La clase Publicacion contiene los datos que necesita el videojuego para representar y modificar una publicacion.

Listing 2: Clase Publicacion.

```
1 class Publicacion:
2     def __init__(self, id, texto, verdadera, impacto):
3         self.id = id
4         self.texto = texto
5         self.verdadera = verdadera
6         self.impacto = impacto
7
8         self.compartidos = 0
9         self.verificaciones = 0
10        self.reportes = 0
11
```

```
12 self.estado = "ACTIVA"
```

Atributo	Funcion
id	Llave de ordenamiento del ABB.
texto	Contenido de la publicacion.
verdadera	Estado de veracidad definido en el modelo del juego.
impacto	Nivel de impacto de la publicacion.
compartidos	Cantidad de veces compartida.
verificaciones	Cantidad de verificaciones.
reportes	Cantidad de reportes.
estado	Estado actual de la publicacion.

Cuadro 1: Atributos utilizados por Publicacion.

4 Operaciones del ABB

4.1 Insercion - insertar()

La insercion crea un nuevo nodo. Si el arbol esta vacio, el nuevo nodo se convierte en la raiz. En caso contrario, se compara el ID nuevo con el ID del nodo actual: si es menor se avanza por la izquierda y si es mayor por la derecha.

Listing 3: Operacion de insercion.

```
1 def insertar(self, publicacion):
2     nuevo = Nodo(publicacion)
3
4     if self.raiz is None:
5         self.raiz = nuevo
6         return
7
8     actual = self.raiz
9
10    while True:
11        if publicacion.id < actual.publicacion.id:
12            if actual.izquierda is None:
13                actual.izquierda = nuevo
14                return
15            actual = actual.izquierda
16
17        elif publicacion.id > actual.publicacion.id:
18            if actual.derecha is None:
19                actual.derecha = nuevo
20                return
21            actual = actual.derecha
22
23    else:
24        return
```

En la aplicacion la insercion es progresiva. Una publicacion entra al ABB cuando aparece en el flujo de la partida.



Figura 2: Nueva publicacion visible en el feed de Civitas antes de ser insertada en el ABB.

4.2 Búsqueda - `buscar()`

La búsqueda parte desde la raíz y utiliza el mismo criterio de orden del ABB. Si el ID buscado es menor, se recorre el subarbol izquierdo; si es mayor, el derecho. Si coincide, se devuelve la publicacion.

Listing 4: Operacion de busqueda.

```

1 def buscar(self, id):
2     actual = self.raiz
3
4     while actual is not None:
5         if id == actual.publicacion.id:
6             return actual.publicacion
7
8         elif id < actual.publicacion.id:
9             actual = actual.izquierda
10
11        else:
12            actual = actual.derecha
13
14    return None

```

En el juego, antes de procesar una decision se utiliza `buscar()` para localizar la publicacion que el jugador esta tratando.

4.3 Eliminacion - `eliminar()`

La eliminacion se realiza buscando primero el nodo y luego resolviendo uno de tres casos: nodo hoja, nodo con un solo hijo o nodo con dos hijos.

Listing 5: Operacion de eliminacion.

```

1 def eliminar(self, id):
2     self.raiz = self._eliminar_recurativo(self.raiz, id)
3
4 def _eliminar_recurativo(self, nodo, id):

```

```

5  if nodo is None:
6      return None
7
8  if id < nodo.publicacion.id:
9      nodo.izquierda = self._eliminar_recurso(
10         nodo.izquierda, id
11     )
12
13 elif id > nodo.publicacion.id:
14     nodo.derecha = self._eliminar_recurso(
15         nodo.derecha, id
16     )
17
18 else:
19     if nodo.izquierda is None and nodo.derecha is None:
20         return None
21
22     if nodo.izquierda is None:
23         return nodo.derecha
24
25     if nodo.derecha is None:
26         return nodo.izquierda
27
28     sucesor = self._encontrar_minimo(nodo.derecha)
29     nodo.publicacion = sucesor.publicacion
30     nodo.derecha = self._eliminar_recurso(
31         nodo.derecha, sucesor.publicacion.id
32     )
33
34 return nodo

```

4.4 Casos de eliminacion

Hoja: se reemplaza el nodo por None.

Un hijo: el nodo es reemplazado por su hijo.

Dos hijos: se utiliza el sucesor minimo del subarbol derecho y se reorganiza el arbol.



Figura 3: Eliminacion de la publicacion #60 del ABB mediante `eliminar()` tras la accion Reportar.

5 Recorridos

La estructura implementa los tres recorridos solicitados para esta entrega.

5.1 Inorden

Visita izquierda, nodo y derecha. En un ABB, produce las llaves en orden ascendente.

Listing 6: Recorrido inorden.

```
1 def inorden(self):
2     resultado = []
3     self._inorden_recursivo(self.raiz, resultado)
4     return resultado
5
6 def _inorden_recursivo(self, nodo, resultado):
7     if nodo is not None:
8         self._inorden_recursivo(nodo.izquierda, resultado)
9         resultado.append(nodo.publicacion)
10        self._inorden_recursivo(nodo.derecha, resultado)
```

5.2 Preorden

Visita nodo, izquierda y derecha.

Listing 7: Recorrido preorden.

```
1 def preorden(self):
2     resultado = []
3     self._preorden_recursivo(self.raiz, resultado)
4     return resultado
5
6 def _preorden_recursivo(self, nodo, resultado):
7     if nodo is not None:
8         resultado.append(nodo.publicacion)
9         self._preorden_recursivo(nodo.izquierda, resultado)
10        self._preorden_recursivo(nodo.derecha, resultado)
```

5.3 Postorden

Visita izquierda, derecha y nodo.

Listing 8: Recorrido postorden.

```
1 def postorden(self):
2     resultado = []
3     self._postorden_recursivo(self.raiz, resultado)
4     return resultado
5
6 def _postorden_recursivo(self, nodo, resultado):
7     if nodo is not None:
8         self._postorden_recursivo(nodo.izquierda, resultado)
9         self._postorden_recursivo(nodo.derecha, resultado)
10        resultado.append(nodo.publicacion)
```

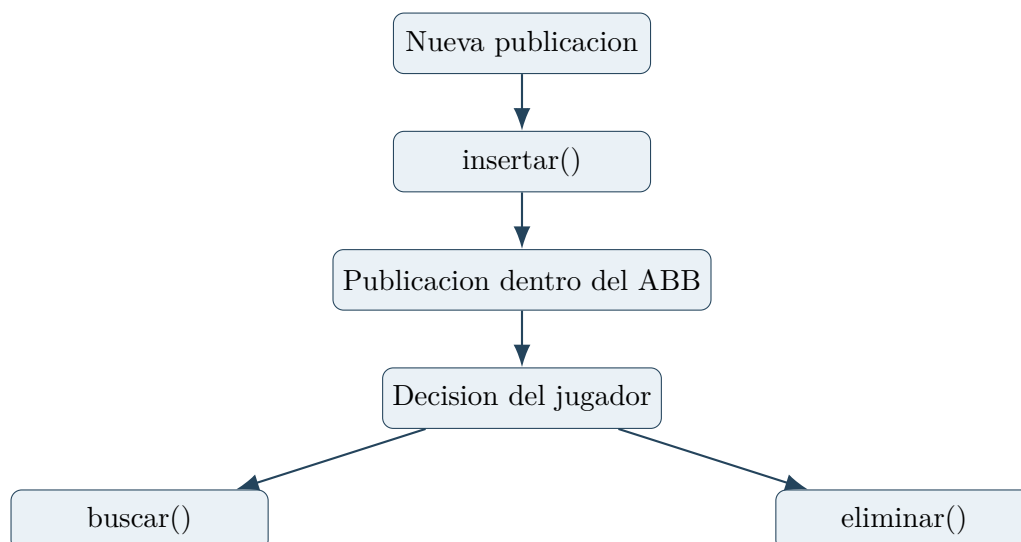


Figura 4: Visualización del ABB con los botones de recorridos, mostrando el resultado del recorrido Inorden.

6 Relacion del arbol con la logica del juego

Esta es la parte central de la primera entrega. El ABB no esta separado de la historia: las publicaciones del videojuego son los elementos almacenados en la estructura y las decisiones del jugador modifican su estado.

6.1 Flujo principal



6.2 Acciones del jugador

La logica actual utiliza cuatro acciones.

Compartir. La publicacion se localiza con `buscar()`, aumenta su contador de compartidos e impacto y cambia a estado COMPARTIDA.

Verificar. Se utiliza `buscar()`, aumenta el numero de verificaciones y se actualiza el estado segun el atributo `verdadera`.

Ignorar. La publicacion se localiza con `buscar()` y permanece dentro del ABB con estado `IGNORADA`.

Reportar. La publicacion se localiza y despues se elimina del arbol mediante `eliminar()`.

Listing 9: Integracion de las acciones con el ABB.

```
1 def ejecutar_accion(self, accion):
2     publicacion_actual = self.obtener_publicacion_actual()
3
4     if publicacion_actual is None:
5         return False
6
7     publicacion = self.arbol.buscar(
8         publicacion_actual.id
9     )
10
11    if publicacion is None:
12        return False
13
14    self.ultima_accion = accion
15
16    if accion == "compartir":
17        publicacion.compartidos += 1
18        publicacion.impacto += 1
19        publicacion.estado = "COMPARTIDA"
20
21    elif accion == "verificar":
22        publicacion.verificaciones += 1
23        publicacion.estado = (
24            "VERIFICADA"
25            if publicacion.verdadera
26            else "FALSA VERIFICADA"
27        )
28
29    elif accion == "ignorar":
30        publicacion.estado = "IGNORADA"
31
32    elif accion == "reportar":
33        id_eliminado = publicacion.id
34        self.arbol.eliminar(id_eliminado)
35        self.publicacion_actual_id = None
36
37    return True
```



Figura 5: Pantalla de decision con las opciones Compartir, Verificar, Ignorar y Reportar sobre una publicacion.

7 Interfaz grafica preliminar

La interfaz fue desarrollada en PyQt6. El laboratorio exige que el funcionamiento del juego se muestre graficamente, incluyendo ciudad, personajes, publicaciones y opciones de interaccion. [1]

En esta primera entrega se presenta una interfaz de tipo novela visual con fondos, personajes, dialogos y una pantalla especifica para visualizar el ABB.

7.1 Elementos principales

- Escenarios y fondos de Ciudad Nova.
- Personajes Ciudadano y Periodista.
- Dialogo cinematografico con franjas negras.
- Aparicion y desaparicion de personajes mediante animaciones.
- Enfoque visual del personaje que habla.
- Publicacion visible y opciones de decision.
- Visualizacion grafica del ABB.
- Botones para Inorden, Preorden y Postorden.



Figura 6: Escena con los personajes Ciudadano y Periodista discutiendo una publicación de Civitas.



Figura 7: Escena de decisión sobre la publicación #70, con impacto 3.



Figura 8: Visualizacion grafica del Arbol Binario de Busqueda dentro de la interfaz del juego.

8 Pruebas realizadas

Se realizaron pruebas de las operaciones principales y de su relacion con las acciones del videojuego.

Prueba	Resultado esperado	Resultado observado
Insertar una publicacion	El nodo se incorpora segun su ID	Correcto
Buscar un ID existente	Devuelve la publicacion	Correcto
Buscar un ID inexistente	Devuelve None	Correcto
Eliminar una hoja	El nodo desaparece	Correcto
Eliminar un nodo con un hijo	El hijo ocupa su posicion	Correcto
Eliminar un nodo con dos hijos	Se reorganiza mediante sucesor	Correcto
Inorden	IDs en orden ascendente	Correcto
Preorden	Recorrido raiz-izquierda-derecha	Correcto
Postorden	Recorrido izquierda-derecha-raiz	Correcto
Reportar	La publicacion sale del ABB	Correcto

Cuadro 2: Pruebas principales de la primera entrega.



Figura 9: Resultado de la prueba de búsqueda: la publicación #80 es encontrada con `buscar()` y actualizada.

9 Alcance de esta entrega

El documento del laboratorio establece que la segunda entrega evaluara la implementacion y visualizacion de grafos, sus operaciones y la integracion entre grafos, arboles y mecanica del juego. La entrega final incorporara posteriormente los componentes restantes del producto, entre ellos aleatoriedad, inclusividad, ayuda, elementos motivadores y comunicacion cliente-servidor. [1]

Por tanto, esta documentacion se concentra exclusivamente en los elementos evaluados en la primera entrega.

10 Conclusiones

En esta primera entrega se implemento un Arbol Binario de Busqueda utilizando el ID de las publicaciones como llave. Se desarrollaron las operaciones de insercion, busqueda y eliminacion, junto con los recorridos inorden, preorden y postorden.

La estructura esta integrada con la logica del videojuego: las publicaciones se almacenan en el ABB, las decisiones del jugador utilizan la operacion de busqueda y la accion de reportar elimina una publicacion de la estructura. La interfaz en PyQt6 permite visualizar tanto la interaccion del jugador como el funcionamiento del arbol.

De esta forma, el ABB cumple una funcion dentro de la experiencia del videojuego y puede ser demostrado mediante operaciones y resultados visibles, que es uno de los objetivos centrales de esta primera entrega. [1]

11 Datos de entrega

Repositorio	https://github.com/SAA01/CIVICA
Lenguaje	Python
GUI	PyQt6
Estructura evaluada	Arbol Binario de Busqueda

Referencias

- [1] Universidad del Norte, Departamento de Ingenieria de Sistemas y Computacion, *Laboratorio - Alcalde Digital, Estructura de Datos II*, 2026.